AWS_INFRASTRUCTURE_AUDIT

NetworkPeer AWS Infrastructure Audit

> Service connectivity and architecture health check — September 6, 2026

Service Status Summary

Service	Status	Endpoint	
Web Frontend (Vercel)	✅ LIVE	https://network-peer-web.vercel.app	
GitHub (rudraaxl)	✅ LIVE	https://github.com/rudraaxl/NetworkPeer	
GitHub (addy9087)	✅ LIVE	https://github.com/addy9087/Networkpeer	
API (Vercel)	❌ NOT DEPLOYED	https://network-peer-api-alpha.vercel.app	
API (AWS ECS)	⚠️ UNKNOWN	Via ALB / custom domain	
Android APK	✅ BUILT	~/Desktop/NetworkPeer_Demo/NetworkPeer-v0.1.0-dev.apk	

---

1. Web Frontend

URL: https://network-peer-web.vercel.app
Status: ✅ Deployed and serving
Response: HTTP 200
Build: Vite + TanStack Start, deployed via Vercel

Configuration

- Framework: TanStack Start (Vite-based SSR)
- Styling: Tailwind CSS v4
- Routing: TanStack Router
- State: React Query
- API: Connects to backend via `VITE_API_BASE_URL` env var

Known Issues

- API backend URL defaults to `http://localhost:3000/api/v1` when env var not set
- Production deployment uses env vars configured in Vercel dashboard

---

2. Backend API

Current State

The API was originally deployed to Vercel at `network-peer-api-alpha.vercel.app` but that deployment no longer exists. The production API runs on **AWS ECS Fargate** behind an Application Load Balancer.

Architecture (from Terraform)

```
Internet → ALB (HTTPS) → ECS Fargate Cluster
                            ├── API Service (Fastify)
                            ├── Worker Service (background jobs)
                            └── Migrator (one-shot DB migrations)

Database: PostgreSQL (RDS) with PostGIS
Cache: Redis (ElastiCache)
Auth: AWS Cognito
Storage: S3 (evidence uploads)
```

Terraform Configuration

- **Location**: `infra/terraform/`

- **State**: S3 + DynamoDB lock
- **CI/CD**: GitHub Actions `ecs-release.yml` (manual trigger on main)
- **Environments**: staging, production (via GitHub Environments)

Required AWS Resources

Resource	Purpose	Status
ECS Cluster	Container orchestration	Configured in Terraform
RDS PostgreSQL	Primary database with PostGIS	Configured in Terraform
ElastiCache Redis	Session cache, queues	Configured in Terraform
S3 Bucket	Evidence file storage	Configured in Terraform
Cognito User Pool	Authentication	Configured in Terraform
ALB	HTTPS load balancing	Configured in Terraform
ACM Certificate	TLS for custom domain	Configured in Terraform

Environment Variables Required

```
# Database
DATABASE_URL=postgresql://...

# Redis
REDIS_URL=redis://...

# Auth
COGNITO_USER_POOL_ID=us-east-1_...
COGNITO_CLIENT_ID=...

# Storage
EVIDENCE_BUCKET_NAME=networkpeer-evidence-...
AWS_REGION=us-east-1

# Payment
STRIPE_SECRET_KEY=sk_...
STRIPE_WEBHOOK_SECRET=whsec_...
```

3. AWS CLI Verification Commands

To verify AWS infrastructure (requires AWS CLI + credentials):

```
# Install AWS CLI
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -
        o "awscliv2.zip"
unzip awscliv2.zip && sudo ./aws/install

# Configure credentials
aws configure
# Enter: Access Key ID, Secret Access Key, Region (us-east-1),
        Output format (json)

# Verify ECS services
aws ecs list-clusters
aws ecs describe-services --cluster <cluster-name> --services
        <api-service> <worker-service>

# Verify RDS
aws rds describe-db-instances --db-instance-identifier
        networkpeer

# Verify ElastiCache
aws elasticache describe-cache-clusters

# Verify S3
aws s3 ls s3://networkpeer-evidence-*

# Verify Cognito
aws cognito-idp list-user-pools --max-results 10

# Verify ALB
aws elbv2 describe-load-balancers --names networkpeer-api

# Test API health (once you have the ALB DNS or custom domain)
curl -s https://<api-domain>/api/v1/health
curl -s https://<api-domain>/api/v1/live
```

---

4. GitHub Actions CI/CD

Workflows

Workflow	Trigger	Status	Purpose
`android.yml`	PR/push to main (android paths)	✅ Passing	Android unit tests

Workflow	Trigger	Status	Purpose
`foundations.yml`	PR/push to main (all paths)	✅ Passing	Swift tests + Terraform validation
`deploy.yml`	PR/push to main (NetworkPeer-main)	⚠️ Needs fixing	API lint/typecheck/test + Docker build
`ecs-release.yml`	Manual dispatch on main only	✅ Fixed	ECS deployment pipeline
`terraform-plan.yml`	PR (infra paths) / manual	✅ Passing	Terraform plan
`terraform-apply.yml`	Manual dispatch on main only	✅ Fixed	Terraform apply

Recent Fixes Applied

- **`ecs-release.yml`** : Added `branches: [main]` to `workflow_dispatch` trigger to prevent feature-branch execution
- **`deploy.yml`** : Added `branches: [main]` to `workflow_dispatch` trigger
- **`terraform-apply.yml`** : Added `branches: [main]` to `workflow_dispatch` trigger

Security Scan Results

```
npm audit --audit-level=high:
  29 vulnerabilities (19 moderate, 10 high, 0 critical)

  High severity:
  - fast-uri: SSRF via malformed IPv6 normalization
  - image-size: DoS via infinite loops in ICNS/JXL/HEIF parsers

  Moderate severity:
  - @xmldom/xmldom: XML fragment injection
  - fastify: Schema validation bypass
  - decode-uri-component: DoS via malformed percent-encoding

  All fixable via npm audit fix (some require --force for
breaking changes)
```

---

5. Architecture Health

Contracts-First Development ✅

- `packages/contracts/src/index.ts` — Shared TypeScript types
- Types: Job, WorkerProfile, Submission, QualityCheckResult, ReviewEvent, JobAssignment
- Builds with: `tsc -p tsconfig.json`
- All consumers import from `@networkpeer/contracts`

Code Quality

- Web: ESLint 9 + Prettier + TypeScript strict mode
- Android: Kotlin Compose + Material 3
- iOS: Swift Package Manager
- Backend: ESLint + TypeScript strict mode

Build Status

Component	Build	Status
contracts	`tsc`	✅ Passes
web	`vite build`	⚠️ Hangs on large node_modules (macOS APFS issue)
Android	`gradlew assembleDebug`	✅ Builds in ~4min
iOS	`swift test`	✅ Passes (CI)
Backend	`npm run build`	✅ Passes (CI)

---

6. Recommendations

1. **Deploy API to Vercel or verify ECS deployment** — The `network-peer-api-alpha.vercel.app` endpoint is down
2. **Fix node_modules issue** — Run `rm -rf node_modules && npm install --legacy-peer-deps` in `NetworkPeer-vip-main`
3. **Address npm audit** — Run `npm audit fix` for moderate vulnerabilities, `npm audit fix --force` for high (with testing)
4. **Install AWS CLI** — Needed for infrastructure verification

5. **Set up Cognito** — Required for authentication flow
6. **Configure Stripe** — Required for escrow payments